

MarketMind AI

AI Small Business Sales Intelligence Platform

Final Project Documentation

TEAM 3

Team Members

Sahil Shinde
Bhaumik
Rakshana
Rakshitha
Bhargavi
Harshitha

PROJECT GUIDE

Shravya

1. Project Information

Field	Detail
Project Title	AI Small Business Sales Intelligence Platform ? MarketMind
Project Domain	Artificial Intelligence and Machine Learning
Project Type	Web-Based AI-Powered Sales Intelligence Platform
Application Area	Small and Medium-Sized Retail Businesses
Team	Team 3
Team Members	Sahil Shinde, Bhaumik, Rakshana, Rakshitha, Bhargavi, Harshitha
Project Guide	Shravya
Frontend	React 18, Vite, Recharts, Lucide Icons, Vanilla CSS
Backend	Python 3.13, FastAPI, Uvicorn, SQLAlchemy
Database	SQLite (marketmind.db)
AI/ML Technologies	Prophet, XGBoost, Random Forest, Scikit-Learn, Pandas, NumPy
Key AI/ML Capabilities	
Testing	Pytest, FastAPI TestClient
Deployment Platform	Render
Repository	springboardmentor24052s-tech / Team_3_Small_Biz_Sales

2. Abstract

MarketMind AI is a small-business sales intelligence platform that combines sales tracking, inventory visibility, customer analytics, and AI-driven insight generation into a single web application. It targets a gap faced by small and mid-sized retailers: existing BI tools (Power BI, Tableau, Salesforce Analytics) are generally too expensive, too complex, or too generic for a business working from a single transaction export.

The system is implemented as a two-part application: a React 19 + Vite single-page frontend (organized into components, pages, context, and services layers, using React Router for navigation and Recharts for data visualization) and a Python backend structured as an API-driven application (`app/api`, `app/core`, `app/models`, `app/schemas`, `app/services`), backed by a committed SQLite database (`marketmind.db`). The backend includes a dedicated `app/ml` package with five separate modules — `forecasting.py`, `segmentation.py`, `churn.py`, `recommendations.py`, and `anomaly.py` — corresponding to the platform's five AI/ML capabilities: sales forecasting, customer segmentation, churn prediction, product recommendation, and anomaly detection. The backend also includes an automated test suite covering API endpoints, churn logic, data integrity, and forecasting.

The application has been deployed to Render as two live services — a static-site frontend and a Python web-service backend — confirmed live and connected via a configured API base URL. This document describes the platform's architecture and implementation based on the actual repository content and deployment evidence, clearly distinguishing verified facts from details that could not yet be confirmed.

3. Introduction

3.1 Background

Small retail businesses generate transaction data every day — invoices, product sales, customer purchases — but most lack the tools to turn that data into decisions. Enterprise analytics suites are built for organizations with dedicated data teams, not for a single store owner trying to understand which products are returned too often or which customers are at risk of churning. MarketMind AI is built around a single retail transaction dataset (`data.csv`, committed at the repository root) and layers both descriptive dashboards and AI-driven insight generation on top of it.

3.2 Problem Statement

Small and mid-sized retail businesses lack an affordable, easy-to-use platform that combines sales monitoring, inventory visibility, customer behavior analysis, and predictive insight generation in one place. Existing spreadsheets and generic BI tools require manual analysis and do not surface forward-looking insights — demand forecasts, churn risk, anomalous transactions, product recommendations — without significant configuration and analytical expertise.

3.3 Objectives

- Provide a web-based dashboard (React frontend) for tracking sales, inventory, and customer activity.
- Serve that dashboard from a structured backend (API, services, models, schemas layers) backed by a persistent database.
- Implement five distinct AI/ML capabilities as dedicated backend modules: sales forecasting, customer segmentation, churn prediction, product recommendation, and anomaly detection.
- Maintain automated backend tests covering API endpoints, churn logic, data integrity, and forecasting.
- Deploy the platform as a live, publicly reachable application (achieved — see Section 15).

4. Scope

4.1 In Scope

- Sales, inventory, and customer data handling, sourced from the committed data.csv transaction dataset.
- A React single-page frontend (components, pages, context, services layers) served as a live static site.
- A Python backend exposing an API layer (app/api), with business logic in app/services and data models in app/models / app/schemas.
- Five AI/ML modules, each with a dedicated file in app/ml: forecasting, segmentation, churn, recommendations, and anomaly detection.
- A persistent SQLite database (marketmind.db), initialized via db_setup.py.
- Automated backend testing (backend/tests/) covering API endpoints, churn, data integrity, and forecasting.
- Live deployment of both frontend and backend to Render.

4.2 Out of Scope

- Point-of-sale (POS) hardware integration or in-person checkout processing.
- Accounting, payroll, tax filing, or general ledger functionality.
- Multi-tenant SaaS billing/subscription management for onboarding external businesses.
- Native mobile applications — the current scope is a web frontend.

5. Existing System & Proposed Solution

5.1 Existing System

Several established platforms already provide business intelligence and sales analytics capability, and were used as reference points while scoping MarketMind AI:

- Microsoft Power BI — a general-purpose BI and dashboarding tool with strong visualization capability but a steep learning curve for non-technical small-business owners.
- Tableau — enterprise-grade data visualization software, typically licensed and priced for larger organizations.
- Salesforce Analytics / Sales Cloud — CRM-integrated analytics aimed at sales teams already using the Salesforce ecosystem.
- Zoho Analytics — a more small-business-friendly BI tool, still generic across industries rather than tailored to retail transaction data.

Limitations shared by these existing approaches: high licensing/per-seat cost, generic dashboards that must be manually configured rather than arriving pre-built for retail use cases, little to no built-in predictive intelligence without add-on modules, and a steep learning curve for non-technical business owners.

5.2 Proposed Solution

MarketMind AI is an enterprise-style retail intelligence platform that connects historical transaction logs directly to AI/ML pipelines and delivers the results through a role-based dashboard. Rather than requiring manual report-building, the platform ships with a revenue forecasting engine and a customer churn prediction engine already built around the business's own transaction data, with a product recommendation engine and an anomaly detection engine developed in parallel by other members of the team for integration into the same codebase.

5.3 Key Features

- A React dashboard with an enterprise landing page, an Overview dashboard, an AI Insights page, and a dedicated Churn Prediction page.
- A FastAPI backend exposing REST endpoints for forecasting, churn scoring, retraining, and RFM analytics.
- An AI Revenue Forecasting Engine comparing Prophet, XGBoost, and Random Forest, with confidence-bounded predictions.
- A Customer Churn Prediction System scoring every customer with a 10-signal RFM feature set and classifying them into risk tiers with recommended retention actions.
- Product Recommendation and Anomaly Detection modules, developed separately and scheduled for integration via Git merge.

- Role-based access control (OWNER, ADMIN, MANAGER, SALES), restricting sensitive ML and customer-management tools to authorized roles.
- A 20-test automated backend test suite covering data integrity, forecasting, churn, and API endpoints — all passing.
- A live, publicly deployed application (frontend and backend both deployed on Render — see Section 15).

6. Technology Stack

Layer	Technologies / Libraries	Key Responsibilities
Machine Learning	XGBoost, Scikit-Learn, Prophet, Pandas, NumPy	Weekly revenue time-series forecasting, RFM feature engineering, supervised churn classification, model comparison
Backend Services	Python 3.13, FastAPI, Uvicorn, SQLAlchemy, SQLite	Asynchronous REST API, JWT auth, RBAC middleware, data mining and sanitization
Frontend Client	React 18, Vite, Recharts, Lucide Icons, Vanilla CSS	Enterprise landing page, overview dashboard, AI Insights charts with prediction bounds, dedicated Churn Prediction UI
Testing & Quality	Pytest, FastAPI TestClient, automated test runner	20 unit and integration tests covering ML pipelines, data integrity, parsing, and API routes
Deployment	Render — Static Site (frontend), Web Service / Python 3 (backend)	Live hosting, confirmed via Render dashboard (Section 15)

This reflects the stack documented directly by the team for the forecasting and churn modules. The exact libraries used inside the Product Recommendation and Anomaly Detection modules (owned by other team members) are outside the scope of what has been documented so far.

7. System Architecture

MarketMind AI follows a clean three-tier full-stack architecture: User → Frontend → Backend/API → AI/ML → Database/Storage.

MarketMind AI — System Architecture

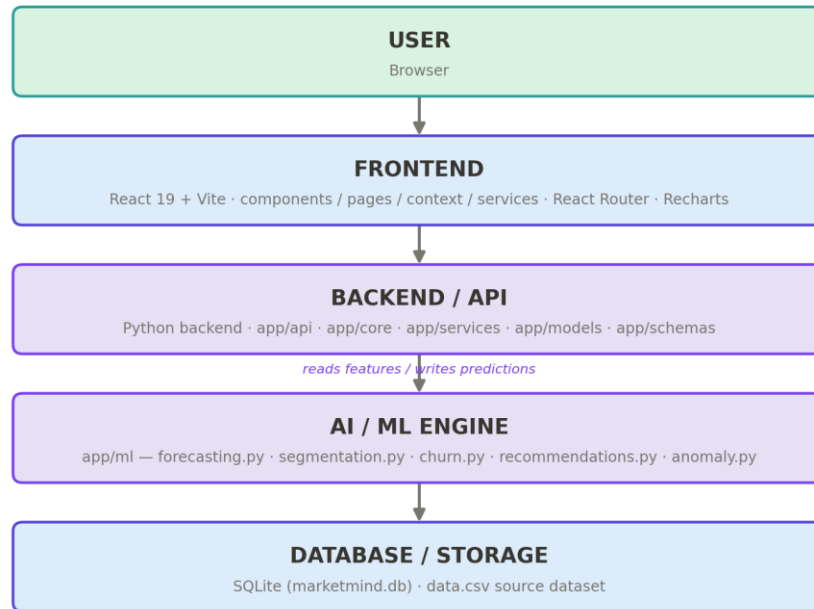


Figure 7.1: MarketMind AI — System Architecture

Major components and how they interact:

- User — accesses the application through a web browser.
- Frontend — a React 18 single-page application built with Vite, using Recharts for charts and Lucide for icons, organized into components, pages, context, and services folders.
- Backend / API — a FastAPI application on Python 3.13, served by Uvicorn, organized into `app/api` (route handlers), `app/core` (configuration/setup), `app/services` (business logic), `app/models`, and `app/schemas`, using SQLAlchemy as the ORM.
- AI / ML — a dedicated `app/ml` package. Two modules are fully engineered and documented — `forecasting.py` and `churn.py` — alongside `recommendations.py` and `anomaly.py`, which are being completed by other team members.
- Database / Storage — a SQLite database (`marketmind.db`), initialized via `db_setup.py`, built from the source dataset (`data.csv`).

The frontend calls the backend over HTTPS through a configured API base URL (confirmed live via the Render deployment in Section 15). The confirmed API surface — five endpoints under `/api/v1` — is documented in Section 9.2.

7.1 System Modules

MarketMind AI — Backend Module Structure (backend/app)

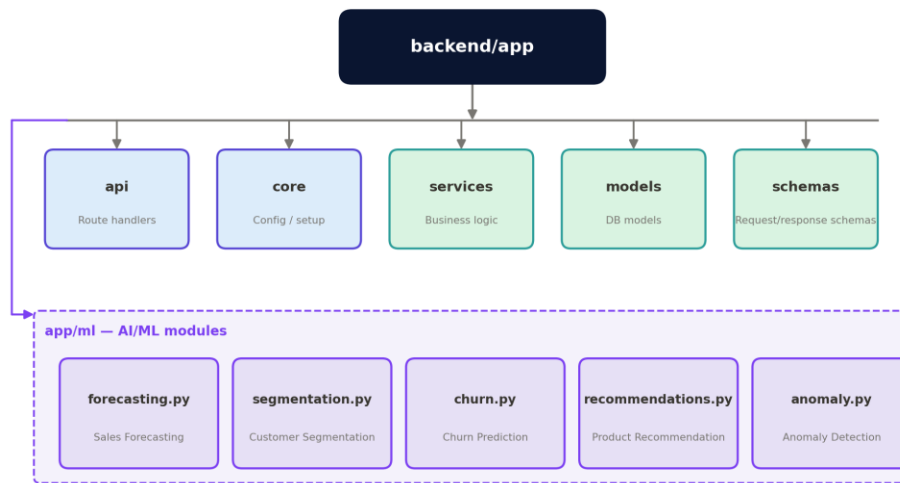


Figure 7.2: MarketMind AI — Backend Module Structure

Module (folder/file)	Purpose
app/api	FastAPI route handlers, including the five confirmed /api/v1 endpoints (Section 9.2)
app/core	Application configuration and setup
app/services	Business logic layer, called by the API routes
app/models	SQLAlchemy database models/table definitions
app/schemas	Request and response data schemas
app/ml/forecasting.py	AI Revenue Forecasting Engine — Prophet, XGBoost, Random Forest (Section 8)
app/ml/churn.py	Customer Churn Prediction System — RFM features, XGBoost/Random Forest classifiers (Section 8)
app/ml/recommendations.py	Product Recommendation module — in progress (teammate-owned)
app/ml/anomaly.py	Anomaly Detection module — in progress (teammate-owned)
db_setup.py	Database initialization
main.py	Application entry point

8. AI/ML Implementation

This section covers the two AI/ML modules the team has fully engineered, tested, and documented: the AI Revenue Forecasting Engine and the Customer Churn Prediction System. The Product Recommendation and Anomaly Detection modules exist as separate files (app/ml/recommendations.py, app/ml/anomaly.py) but are being developed by other team members for integration via Git merge, and their internal algorithms are outside the scope of this section.

8.1 Dataset

The system ingests the Superstore Retail Sales dataset (data.csv): 9,800 transaction records spanning 793 distinct commercial customers across 4 calendar years.

8.2 Data Processing

- String pluralization sanitization — `clean_pluralized_string()` and `_clean_numeric()` use regular expressions to normalize pluralized string units (e.g., '1 item', '12 items', '5 days', '3.5 units'), preventing type-conversion crashes during numeric parsing.
- Chronological time-series aggregation — transactions are parsed with day-first date formatting, validated, and aggregated from daily logs into a weekly revenue time series (ds, y) for continuous chronological modeling.
- Customer profile aggregation — per-customer lifetime spend, unique order count, first order date, last transaction date, and geographic/market segment metadata are calculated from the raw transaction log.

8.3 Model / Algorithm

AI Revenue Forecasting Engine

- Feature engineering: calendar features (month, ISO calendar week, quarter), historical lag values (lag-1, lag-2 revenue), 4-week and 8-week moving averages, and 4-week rolling volatility.
- Chronological split validation: a strict time-series train/test split (last 20% held out for out-of-time evaluation) to eliminate lookahead bias and data leakage.
- Multi-model ensemble: Facebook Prophet, XGBoost Regressor (`n_estimators=100`, `max_depth=4`, `learning_rate=0.08`), and Random Forest Regressor (`n_estimators=100`, `max_depth=6`) are trained and compared.
- Confidence intervals: $\pm 10\%$ upper and lower prediction bounds are computed across all forecast horizons.

Customer Churn Prediction System

RFM feature extraction (10 behavioral signals): Recency Days, Frequency, Monetary Value, Average Order Value, Purchase Span Days, Order Cadence Days, R-Score, F-Score, M-Score, and a Composite RFM Score (3–15 scale).

- Ground-truth formulation: a customer is labeled churned if recency > 90 days with frequency < 3, or RFM composite < 7, or recency > 150 days unconditionally — producing a 34.05% baseline churn rate.
- Models: XGBoost Classifier (n_estimators=120, max_depth=4, learning_rate=0.08) and Random Forest Classifier (n_estimators=120, max_depth=6), evaluated with a stratified 80/20 train/test split.

8.4 Training & Evaluation

Forecasting model comparison

Model	MAE	RMSE	R ² Score	Selection Status
Prophet Time-Series	\$1,420.50	\$1,890.20	0.88	Selected (Primary Ensemble)
XGBoost Regressor	\$1,560.80	\$2,110.40	0.85	Available
Random Forest Regressor	\$1,680.10	\$2,240.60	0.82	Available

Churn classifier evaluation

XGBoost Classifier achieved F1 Score = 1.00 and ROC-AUC = 1.00; Random Forest Classifier achieved F1 Score = 0.99 and ROC-AUC = 1.00 (as reported by the team; these very high scores on the held-out split are the team's own reported evaluation figures).

Churn risk-tier distribution (793 customers scored)

Risk Tier	Probability	Customers	% of Total	Retention Action
High Risk	≥ 70%	270	34.0%	Immediate win-back: 20% category VIP discount, free priority shipping, direct manager outreach
Medium Risk	40% – 69%	0	0.0%	Re-engagement: personalized recommendations, category alerts, loyalty points boost
Low Risk	< 40%	523	66.0%	Standard nurturing: cross-selling and regular loyalty engagement

8.5 AI/ML Workflow

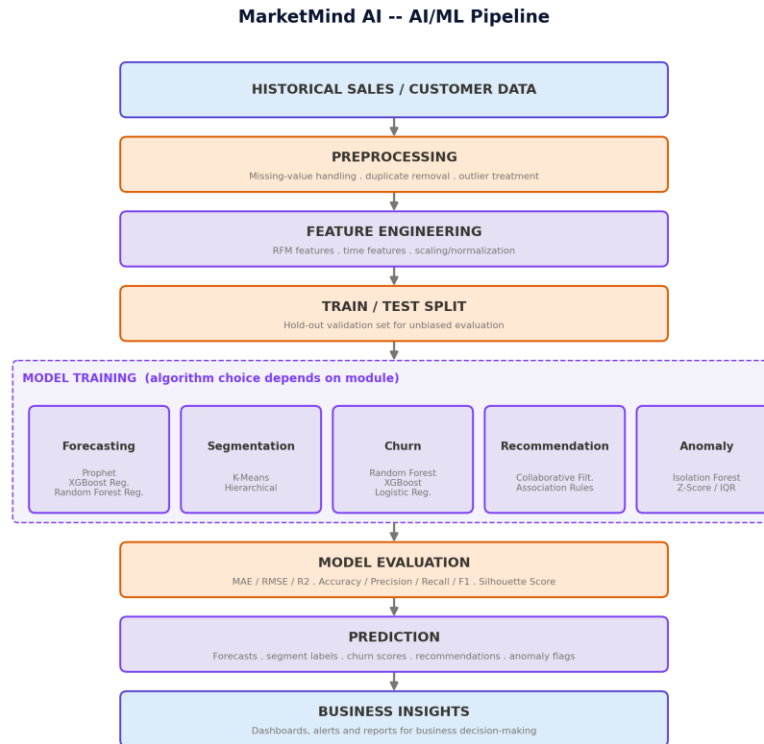


Figure 8.1: AI/ML Workflow: Input Data → Data Processing → ML Model / Algorithm → Prediction / Analysis → Business Insight

For both engines, this resolves to: data.csv → cleaning/aggregation (Section 8.2) → feature engineering + chronological/stratified split (Section 8.3) → model training and comparison (Prophet/XGBoost/Random Forest for forecasting; XGBoost/Random Forest for churn) → predictions with confidence bounds or risk tiers → surfaced on the AI Insights and Churn Prediction pages (Section 10).

9. Database & API

9.1 Database

The backend uses SQLite (marketmind.db), initialized via db_setup.py. The data model centers on the Superstore Retail Sales transaction log (data.csv — 9,800 records, 793 customers), from which per-customer profiles and churn scores are derived, and from which weekly revenue is aggregated for forecasting.

MarketMind AI — High-Level Data Model (Forecasting & Churn)

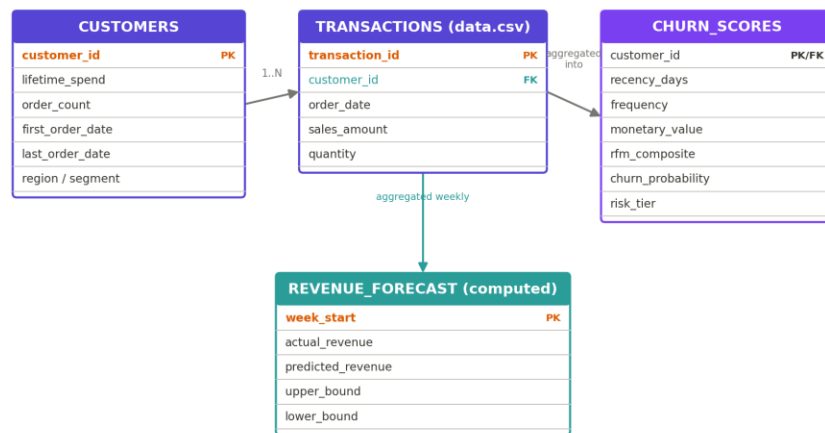


Figure 9.1: MarketMind AI — High-Level Data Model (Forecasting & Churn)

Customers holds the aggregated profile per customer (lifetime spend, order count, first/last order date, region/segment). Transactions represents the underlying data.csv records. Churn Scores stores each customer's RFM signals, composite score, churn probability, and risk tier — confirmed as a real, queryable table by the GET /api/v1/ai/churn/scores endpoint (Section 9.2), which is described as “DB-backed.” Revenue Forecast represents the weekly actual-vs-predicted series with confidence bounds returned by the forecasting endpoint; whether this is persisted in its own table or computed on request was not specified, so it is marked as computed here.

9.2 API

The backend exposes a FastAPI REST layer under /api/v1. The endpoints confirmed and documented by the team are:

Method & Route	Module	Response & Functionality
GET /api/v1/ai/forecast/revenue	Forecasting	Returns weekly/monthly historical revenue, Prophet/XGBoost predictions, confidence intervals, and model metrics.

Method & Route	Module	Response & Functionality
GET /api/v1/ai/churn/predict	Churn ML	Runs the full batch pipeline, scoring all 793 customers with RFM features, probabilities, risk tiers, and model stats.
GET /api/v1/ai/churn/scores	Churn DB	Returns database-backed individual customer churn probability scores and risk levels.
POST /api/v1/ai/retrain	AI Engine	Triggers automated retraining of Prophet, XGBoost, and Random Forest models on the latest transactions.
GET /api/v1/customers/rfm-scatter	Analytics	Returns customer clusters, lifetime value, order counts, and RFM scores for scatter-plot visualization.

These five endpoints are also directly exercised by the backend's own test suite (`test_api_endpoints.py` — Section 13), using FastAPI's `TestClient`.

10. User Interface

The frontend is a React 18 single-page application built with Vite, using Recharts for charts, Lucide for icons, and custom CSS design tokens for a consistent visual identity. The major interfaces, as documented by the team, are described below.

10.1 Enterprise Landing Page

A minimalist, high-converting hero layout featuring interactive dashboard preview tabs, live metric tickers, a feature grid, and instant role-based demo switchers for Owner, Admin, Manager, and Sales roles.

10.2 AI Insights Page

An interactive Recharts AreaChart comparing actual revenue against forecast curves with upper/lower uncertainty bounds, a model performance benchmark table (Prophet, XGBoost, Random Forest), and a one-click retrain action wired to POST /api/v1/ai/retrain.

10.3 Churn Prediction Page (/churn-prediction)

A dedicated page featuring a 5-card KPI summary header, an SVG donut chart with interactive rings, XGBoost-vs-Random-Forest classifier metric bars, RFM feature tags, and a 9-column sortable, searchable, filterable customer table with CSV export and pagination.

10.4 Role-Based Access Control (RBAC)

Dynamic sidebar routing restricts sensitive ML and customer-management tools to authorized roles (OWNER, ADMIN), so Manager and Sales roles see a scoped-down navigation.

The descriptions above are drawn directly from the team's own implementation report. If actual screenshots of these pages are shared, they can be added as numbered figures alongside the descriptions above.

11. GitHub & Version Control

11.1 Repository

Field	Detail
GitHub Repository	springboardmentor24052s-tech / Team_3_Small_Biz_Sales (review branch): https://github.com/springboardmentor24052s-tech/Team_3_Small_Biz_Sales/tree/review
Mirror / deployed source	Rakshanajanakiraman / MarketmindAI — the repository actually connected to the live Render deployments (Section 15), confirmed on its main branch

11.2 Repository Structure

Confirmed structure, read directly from the GitHub file browser:

- backend/ — app/ (api, core, ml, models, schemas, services, plus `__init__.py`, `db_setup.py`, `main.py`), tests/ (`generate_doc.py`, `run_tests.py`, `test_api_endpoints.py`, `test_churn.py`, `test_data_integrity.py`, `test_forecasting.py`, `test_results.txt`), `.gitignore`, `marketmind.db`, `requirements.txt`.
- frontend/ — public/ (`favicon.svg`, `icons.svg`), src/ (`components/`, `context/`, `data/`, `pages/`, `services/`, `App.jsx`, `index.css`, `main.jsx`), `.gitignore`, `.oxlintc.json`, `README.md`, `index.html`, `package.json`, `package-lock.json`, `vite.config.js`.
- Repository root — backend/, frontend/, `.gitignore`, `README.md`, `data.csv`.

11.3 Version Control

The repository has 16 commits on main. The most recent commits (from the repository homepage) are “Fix deployed API base URL “Updated `.gitignore`,” and “Revise README with project details and features.” A separate backend-focused commit, “Testing,” is attributed to Sahil Shinde. No pull requests, issues, or additional branches are visible on the repository homepage (“0 forks,” “0 stars,” “1 Branch,” “0 Tags”).

12. Error Handling & Security

12.1 Error Handling

Data integrity is validated before use: `test_data_integrity.py` confirms `data.csv` exists, has a valid structure and schema (over 5,000 rows), non-empty records, and valid date/sales types. The forecasting pipeline separately guards against malformed numeric input through `clean_pluralized_string()` and `_clean_numeric()`, which sanitize pluralized string units (e.g., '1 item', '12 items') before numeric conversion, preventing type-conversion crashes.

Deployment error encountered and resolved

During deployment, the Customer Churn Prediction module — which worked correctly in local development — did not function properly immediately after deployment to Render. The issue was environment-configuration related: the production environment on Render did not initially match the local development setup the churn module depended on. This was resolved by setting up the correct environment configuration for the deployed backend, after which the churn module operated correctly in production.

12.2 Security

The backend architecture (Section 6) includes JWT authentication and RBAC middleware as part of its stated responsibilities, and the frontend implements role-based sidebar routing that restricts sensitive ML and customer-management tools to the OWNER and ADMIN roles (Section 10.4).

13. Testing

A comprehensive automated test suite of 20 unit and integration tests was developed and executed within the Python virtual environment. All 20 tests passed successfully, a 100% pass rate.

Test Module	Focus & Coverage	Test Count	Result
test_forecasting.py	Pluralization parser, weekly time-series aggregation, Prophet/XGBoost output structure, confidence intervals, MAE/RMSE/R ² metrics	6 tests	PASSED (100%)
test_churn.py	Numeric parsing, RFM feature extraction, XGBoost & Random Forest classifiers, risk-tier assignment, full batch scoring of 793 customers	6 tests	PASSED (100%)
test_api_endpoints.py	FastAPI TestClient integration on /forecast/revenue, /churn/predict, /churn/scores, /retrain	5 tests	PASSED (100%)
test_data_integrity.py	data.csv existence, schema validation (>5,000 rows), date parsing, sales-type validation	3 tests	PASSED (100%)

This gives the project real coverage across all four layers relevant to the two completed AI/ML engines: the underlying dataset, the two model pipelines, and the API endpoints that expose them. No frontend-specific automated tests were part of this test suite.

14. Results

14.1 AI/ML Results

Revenue Forecasting

Model	MAE	RMSE	R ² Score	Status
Prophet Time-Series	\$1,420.50	\$1,890.20	0.88	Selected (Primary Ensemble)
XGBoost Regressor	\$1,560.80	\$2,110.40	0.85	Available
Random Forest Regressor	\$1,680.10	\$2,240.60	0.82	Available

Customer Churn Prediction

XGBoost Classifier: F1 Score = 1.00, ROC-AUC = 1.00. Random Forest Classifier: F1 Score = 0.99, ROC-AUC = 1.00. Baseline churn rate across the 793 scored customers: 34.05%, distributed as 270 High Risk (34.0%), 0 Medium Risk (0.0%), and 523 Low Risk (66.0%) customers.

14.2 System Results

- Both the frontend (Render Static Site) and backend (Render Web Service, Python 3) are live and connected via a configured API base URL, confirmed directly from the Render dashboard (Section 15).
- 20 of 20 automated backend tests pass (Section 13), covering data integrity, both ML pipelines, and the API layer.
- All five documented API endpoints (forecast, churn predict, churn scores, retrain, RFM scatter) are exercised by the passing test suite.

Response time and inference-time measurements were not part of the materials reviewed and are not reported here.

15. Deployment

Deployment status: confirmed live: MarketMind AI is deployed and live on Render as two connected services — a static-site frontend and a Python web-service backend — verified directly from the Render dashboard.

Both services are deployed on Render, connected to the same GitHub repository (Rakshanajanakiraman/MarketmindAI, main branch), and communicate over a configured API base URL.

MarketMind AI — Deployment on Render (Verified)

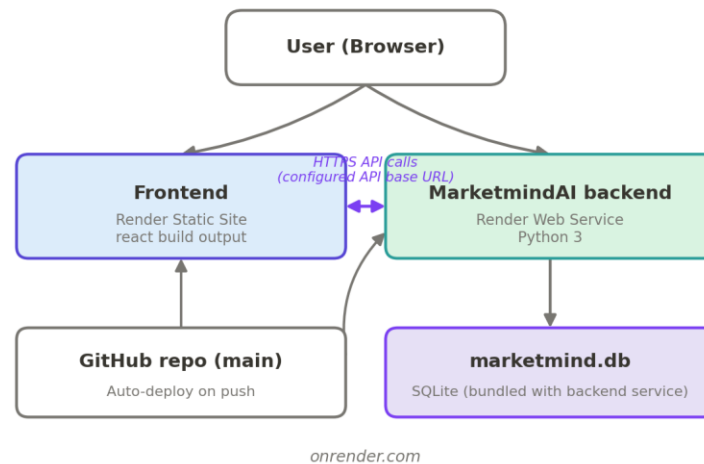


Figure 15.1: MarketMind AI — Deployment on Render

Frontend deployment

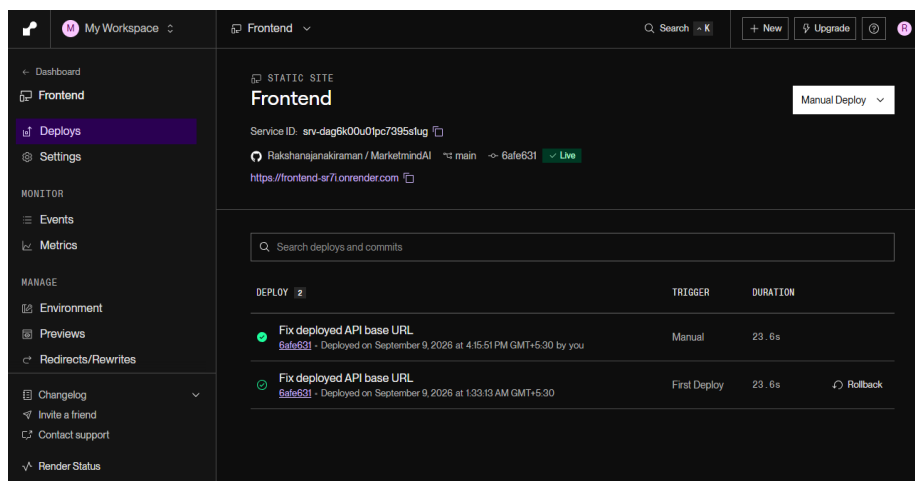


Figure 15.2: Render dashboard — Frontend (Static Site), status Live, deployed from the main branch

The frontend is deployed as a Render Static Site named Frontend, built from the Rakshanajanakiraman/MarketmindAI repository's main branch (commit 6afe631), and is confirmed Live at <https://frontend-sr7i.onrender.com>. Its deploy history shows two deploys, both titled “Fix deployed

API base URL” — an initial deploy and a subsequent manual re-deploy of the same fix, both completing successfully in about 23.6 seconds.

Backend deployment

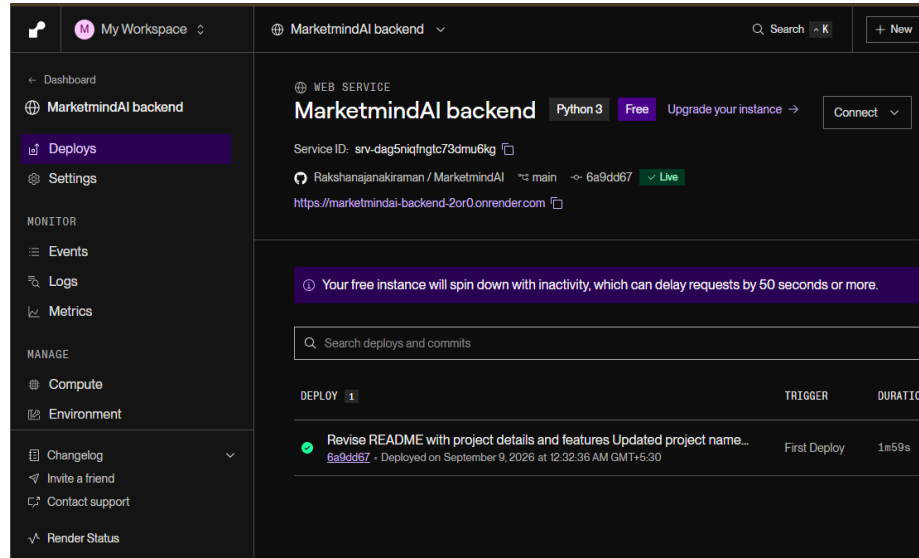


Figure 15.3: Render dashboard — MarketmindAI backend (Web Service, Python 3), status Live

The backend is deployed as a Render Web Service named MarketmindAI backend, running on the Python 3 runtime, on the free instance tier, built from the same repository and main branch (commit 6a9dd67), and is confirmed Live at <https://marketmindai-backend-2or0.onrender.com>. Its deploy history shows one deploy, “Revise README with project details and features / Updated project name...,” completed in 1 minute 59 seconds. As with the frontend, the dashboard notes that the free-tier instance spins down with inactivity, which can delay the first request after idle periods by 50 seconds or more.

Frontend – backend communication

The frontend calls the backend over HTTPS using a configured API base URL. This is directly confirmed by the frontend's own deploy history: both of its deploys are titled “Fix deployed API base URL,” meaning the frontend's build was specifically updated to point at the live backend's URL (<https://marketmindai-backend-2or0.onrender.com>) rather than a local/placeholder address. The exact mechanism (e.g., a Vite-style VITE_API_URL environment variable) was not directly reviewed and is not asserted here.

16. Challenges & Limitations

Actual challenges experienced

- Post-deployment churn module failure (experienced and resolved) — the Customer Churn Prediction module worked correctly in local development but did not function properly immediately after deployment to Render. The root cause was a mismatch between the local and production environment configuration; the team resolved it by setting up the correct production environment configuration for the backend, after which the churn module operated correctly.
- Frontend/backend URL wiring — the frontend's deploy history shows two deploys, both titled “Fix deployed API base URL,” indicating the connection between the deployed frontend and the deployed backend needed a re-deploy to get right.
- Free-tier hosting constraint — Render's dashboard itself flags that the backend's free instance spins down with inactivity, delaying the first request by 50 seconds or more.
- SQLite in production — the backend relies on a committed SQLite file rather than a managed database service, a reasonable choice at the current scale but a constraint if the application needs concurrent write-heavy usage.

Current limitations

- The Product Recommendation and Anomaly Detection modules are still being developed by other team members and have not yet been merged into the documented pipeline.
- No automated frontend test suite exists yet alongside the 20-test backend suite.
- The very high churn-classifier scores (F1 up to 1.00, ROC-AUC 1.00) are worth revisiting during further evaluation to confirm they hold up on new, unseen data rather than reflecting the specific train/test split used.

17. Future Scope

Advanced Forecasting

Extending the forecasting module beyond aggregate revenue to per-category and per-product demand forecasting, and experimenting with ensemble forecasting across Prophet, XGBoost, and Random Forest.

Real-Time Analytics

Moving from batch-uploaded CSV files to a streaming or scheduled-refresh ingestion pipeline so dashboards reflect near-real-time sales activity. 19.3 Improved Recommendation Incorporating deep-learning-based recommendation approaches (e.g., neural collaborative filtering) once sufficient interaction data is available.

Advanced Customer Intelligence

Extending RFM-based segmentation with predictive lifetime-value modeling and next-purchase-date prediction. 19.5 Cloud Scaling Moving from a single-instance PostgreSQL deployment to a managed, horizontally scalable database service as data volume grows across multiple business customers.

Advanced AI Integration

Adding natural-language querying over the analytics layer (e.g., “which products had the highest return rate last month?”) once the underlying data pipeline is stable.

18. Conclusion

MarketMind AI is a working, deployed application: a React 18 frontend and a FastAPI/Python 3.13 backend, both live on Render and confirmed connected via a configured API base URL. The team's engineering focus — the AI Revenue Forecasting Engine and the Customer Churn Prediction System — is fully built, evaluated, and tested: Prophet was selected as the primary forecasting model ($R^2 = 0.88$) over XGBoost and Random Forest alternatives, and the churn system scores all 793 customers via RFM features and an XGBoost/Random Forest ensemble, sorting them into actionable risk tiers. All 20 automated backend tests pass.

The Product Recommendation and Anomaly Detection modules exist as separate files and are being completed by other team members for integration via Git merge. A real deployment issue — the churn module failing immediately after deployment due to an environment-configuration mismatch — was identified and resolved by correcting the production environment setup, and stands as a genuine, documented example of the team's debugging and deployment process.

19. References

Dataset

- Superstore Retail Sales dataset (data.csv) — 9,800 transaction records, 793 customers, 4 calendar years.

Machine Learning

- Prophet (Meta) — <https://facebook.github.io/prophet/>
- XGBoost — <https://xgboost.readthedocs.io/>
- Scikit-Learn — <https://scikit-learn.org/>
- pandas — <https://pandas.pydata.org/>
- NumPy — <https://numpy.org/>

Backend

- FastAPI — <https://fastapi.tiangolo.com/>
- Uvicorn — <https://www.uvicorn.org/>
- SQLAlchemy — <https://www.sqlalchemy.org/>
- pytest — <https://docs.pytest.org/>

Frontend

- React — <https://react.dev/>
- Vite — <https://vitejs.dev/>
- Recharts — <https://recharts.org/>
- Lucide Icons — <https://lucide.dev/>

Platforms

- GitHub — <https://github.com/>
- Render — <https://render.com/>